pyCC.id: A Python package for nonlinear equation discovery based on characteristic curves

25 November 2025

Summary

`pyCC.id` is a Python package for data-driven system identification that provides a flexible "grey-box" framework for discovering interpretable, nonlinear governing equations from time-series data. It is designed to bridge the gap between opaque black-box models (e.g., Neural ODEs) and restrictive, library-dependent methods (e.g., SINDy [1]).

The core philosophy of `pyCC.id` is to decompose a complex dynamical system, $d\mathbf{x}/dt = \mathbf{F}(\mathbf{x}, t)$, into a user-defined model structure, $\mathbf{G}$. This structure explicitly separates measured inputs (like system state $\mathbf{x}$ and external forces) from the unknown model components. These components are: (1) a set of unknown one-dimensional functions ($\{\mathbf{f}\}$) referred to as **Characteristic Curves** (CCs), which capture the underlying system nonlinearities (such as stiffness or damping), and (2) a set of unknown scalar parameters ($\mathbf{a}$).

The package offers a flexible, multi-backend approach to identify these unknown functions and scalar parameters. Users can model the characteristic curves $\{\mathbf{f}\}$ using powerful non-biased approximators like neural networks (NN-CC, via `PyTorch` [2], and compatible with both CPU and GPU devices), simple polynomial basis functions (Poly-CC), or discover analytical expressions directly using symbolic regression (SymbR-CC, via `PySR` [3]). A distinctive feature of this framework is that physical prior knowledge (e.g., symmetries, conservation laws) can be easily and directly incorporated as constraints, guiding the discovery process toward physically consistent models for experts in engineering, physics, and biology.

Statement of Need

Researchers in science and engineering often face a trade-off in system identification. **Black-box models**, such as standard Neural ODEs[4], can achieve high predictive accuracy but are opaque, offering little physical insight[5]. Conversely,

interpretable "white-box" methods, like SINDy [1] or pure Symbolic Regression [3], aim to find simple analytical equations. However, their success often depends on the true dynamics being sparsely represented in a pre-defined library of candidate functions or, in the case of SR, can be computationally challenging and highly sensitive to hyperparameter tuning when applied to full, complex systems.

`pyCC.id` is built to fill this "grey-box" gap, targeting domain experts who possess partial physical knowledge of their system (i.e., the model structure $\mathbf{G}$) but need to discover the specific functional forms of its components (the characteristic curves $\{\mathbf{f}\}$). It addresses the need for a tool that can take advantage of powerful, non-biased and non parametric function approximators (e.g., neural networks) within a physically constrained, interpretable framework, rather than forcing an all-or-nothing choice between accuracy and interpretability.

Comparison to other packages

`pyCC.id` integrates concepts from several existing tools but applies them within a unique, structured framework:

- **SINDy (e.g., `pysindy`)**: SINDy [1] excels when the true dynamics are a sparse combination of terms from a user-provided candidate library. `pyCC.id` differs by not relying on a pre-defined library for the full dynamics. Instead, its NN-CC backend learns the shape of unknown 1D functions, which can then be analyzed, offering flexibility when the underlying functions (e.g., complex friction or stiffness) are not easily represented by simple library terms.
- **Symbolic Regression (e.g., `PySR`)**: While `pyCC.id` uses `PySR` [3] as a backend (SymbR-CC) and a post-processing tool, its application is distinct. Rather than applying SR directly to the full, high-dimensional derivative data (a computationally difficult task), `pyCC.id`'s primary workflow first isolates the unknown components as simple 1D functions (using NN-CC) and *then* applies SR to these much simpler, cleaner 1D curves to find their analytical forms.
- **Black-Box Neural ODEs**: Standard Neural ODE packages learn the entire derivative function $\mathbf{F}$ as a single, monolithic neural network. `pyCC.id` employs a "grey-box" approach, using `PyTorch` [2] to model only the specific, interpretable 1D components $\{\mathbf{f}\}$ within a user-defined physical structure $\mathbf{G}$, making the resulting model inherently interpretable.
- **Physics-Informed Neural Networks** (PINNs)[6] represent a powerful approach to simulate systems (e.g., PDEs), and the incorporation of constraints into the loss function during the neural network training. While excellent for solving or parameterizing known or partially-known governing equations, they are typically less suited for the explicit discovery of the

functional form of complex, coupled system components, which often remains a black-box function within the network itself.

Formalism

For many physical systems, the dynamics are described by a set of first-order ordinary differential equations (ODEs):

$$\frac{d\mathbf{x}}{dt} = \mathbf{F}(\mathbf{x}, t)$$

Here, $\mathbf{x}(t)$ is the system state vector, but the function $\mathbf{F}$ is often a complex "black box," making it difficult to gain physical insight. The core philosophy of `pyCC.id` is to decompose this complex function $\mathbf{F}$ into a combination of simpler, interpretable building blocks. We express this decomposition as:

$$\frac{d\mathbf{x}}{dt} = \mathbf{G}(\mathbf{x}, \mathbf{F}_{ext}(t); \{\mathbf{f}\}, \mathbf{a})$$

Where $\mathbf{x}$ and $\mathbf{F}_{ext}(t)$ are the measured inputs (system state and external forces). The goal is to find the model components: $\{\mathbf{f}\}$, a set of unknown 1D **Characteristic Curves** (e.g., nonlinear stiffness or damping functions), and $\mathbf{a}$, a vector of unknown scalar parameters. The user proposes the **model structure** $\mathbf{G}$, which defines how these components are combined. `pyCC.id` is a Python package that discovers the optimal functions $\{\mathbf{f}\}$ and parameters $\mathbf{a}$ that best fit the observed data.

Features

`pyCC.id` is designed to be a flexible and high-performance tool for researchers and practitioners. Its key features include:

- **Interpretable Models**: Decomposes complex, high-dimensional dynamics into a set of simple, 1D characteristic curves that often have direct physical meaning (e.g., stiffness, damping), and a set of scalar parameters (e.g., mass value).
- **Flexible Function Parametrization**: Supports multiple backends for modeling the characteristic curves, allowing users to choose the right tool for their problem:
  - **Neural Networks (NN-CC)**: Uses `PyTorch` [2] for high-performance, non-biased function approximation.
  - **Polynomials (Poly-CC)**: Provides a simple baseline using polynomial basis functions.
  - **Symbolic Regression (SymbR-CC)**: Uses `PySR` [3] to discover analytical expressions directly.

- **Physics-Informed Discovery**: Allows users to inject domain knowledge as constraints during training (e.g., `'f1 odd'`, `'f2(0)=0'`) or by defining conserved quantities in the loss function. This leads to more robust and physically consistent models.
- **Hardware Acceleration**: Natively supports multicore CPUs and GPUs from both NVIDIA (`cuda`) and Intel (`xpu` via `intel-extension-for-pytorch`) for accelerating neural network training.
- **Built-in Simulator**: Includes a versatile ODE simulator (`pycc.simulate`) compatible with all identified model types for validation and analysis.
- **Comprehensive Documentation**: Provides a Google Colab notebook for a quick start, as well as a full gallery of tutorials and examples in the repository.

Mentions of scholarly publications

The `pyCC.id` package provides a generalized software implementation of the CC-based approaches for system identification. This core methodology was central to methods first introduced for first-order systems [7, 8] and later extended to second-order systems [9, 10]. pyCC.id unifies and extends this approach into a single framework capable of handling higher-order dynamical systems.

Key References

The software package is available at: https://github.com/FedejGon/pyCC.id

Acknowledgements

This work was partially supported by CONICET (Consejo Nacional de Investigaciones Científicas y Técnicas, Argentina). We acknowledge the computational resources from the Clementina XXI supercomputer and CCT-Rosario Computational Center, both managed by the High Performance Computing National System (SNCAD, ME-Argentina), with the support of the Undersecretariat of Science and Technology of Argentina.

References

[1] S. L. Brunton, J. L. Proctor, and J. N. Kutz. "Discovering governing equations from data by sparse identification of nonlinear dynamical systems". In: *Proceedings of the National Academy of Sciences* 113.15 (2016), pp. 3932–3937. DOI: 10.1073/pnas.1517384113.

[2] A. Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG]. URL: https://arxiv.org/abs/1912.01703.

[3] M. Cranmer. "Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl". In: *arXiv preprint arXiv:2305.01582* (2023). eprint: 2305.01582. URL: https://arxiv.org/abs/2305.01582.

[4] R. T. Q. Chen et al. *Neural Ordinary Differential Equations*. 2019. arXiv: 1806.07366 [cs.LG]. URL: https://arxiv.org/abs/1806.07366.

[5] K. Wu. "Automations Black-Box Conundrum: The Interpretability Crisis of Data-Driven System Identification and the Path Forward". In: *Theoretical and Natural Science* 120.1 (July 2025), pp. 25–35. ISSN: 2753-8826. DOI: 10.54254/2753-8818/2025.ad25063. URL: http://dx.doi.org/10.54254/2753-8818/2025.AD25063.

[6] M. Raissi, P. Perdikaris, and G.E. Karniadakis. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. ISSN: 0021-9991. DOI: 10.1016/j.jcp.2018.10.045.

[7] F. J. Gonzalez. "Determination of the characteristic curves of a nonlinear first order system from Fourier analysis". In: *Sci. Rep.* 13.1 (Feb. 2023), p. 1955. DOI: 10.1038/s41598-023-29151-5.

[8] F. J. Gonzalez. "System identification based on characteristic curves: a mathematical connection between power series and Fourier analysis for first-order nonlinear systems". In: *Nonlinear Dyn.* 112.18 (July 2024), pp. 16167–16197. ISSN: 1573-269X. DOI: 10.1007/s11071-024-09890-4.

[9] F. J. Gonzalez and L. P. Lara. "Interpretable neural network system identification method for two families of second-order systems based on characteristic curves". In: *Nonlinear Dyn.* (Sept. 2025). DOI: 10.1007/s11071-025-11744-6.

[10] F. J. Gonzalez. "Symmetry-guided neural network system identification using interpretable characteristic curves and post-symbolic regression". In: (Nov. 2025).